

Flux de données — sharo.fr

Branche **master** — état au 2026-06-08

Trois lectures : D (tableaux + anomalies), C (diagrammes ASCII), B (flux narratif par type)

PARTIE D — Tableaux par zone + observations

Zone 1 — Site public (/ , /sessions, /les-epreuves, /categories-epreuves, /:slug)

Activities

Page frontend	Fichier	Appel	Endpoint	Router	Middleware	Controller
ActivitiesPage	pages/ActivitiesPage.tsx	useFetch	GET /api/activities?limit=12&page=N	routers/activities.router.ts	aucun	activities.controller.ts → getActivities
ActivityDetailPage	pages/ActivityDetailPage.tsx	fetch natif	GET /api/activities/by-slug/:slug	routers/activities.router.ts	aucun	activities.controller.ts → getActivityBySlug
ActivityDetailPage	pages/ActivityDetailPage.tsx	fetch natif	GET /api/sessions?activity_slug=X&status=Scheduled&limit=6	routers/sessions.router.ts	aucun	sessions.controller.ts → getSessions
HomePage	pages/HomePage.tsx	useFetch	GET /api/activities?limit=4	routers/activities.router.ts	aucun	activities.controller.ts → getActivities
DynamicDetailPage	pages/DynamicDetailPage.tsx	fetch natif	GET /api/activities/by-slug/:slug	routers/activities.router.ts	aucun	activities.controller.ts → getActivityBySlug

Sessions

Page frontend	Fichier	Appel	Endpoint	Router	Middleware	Controller
SessionsPage	pages/SessionsPage.tsx	useFetch	GET /api/sessions?limit=12&page=N&sort=date&order=asc[&status=X]	routers/sessions.router.ts	aucun	sessions.controller.ts → getSessions
SessionDetailPage	pages/SessionDetailPage.tsx	fetch natif	GET /api/sessions/:id	routers/sessions.router.ts	aucun	sessions.controller.ts → getSession
HomePage	pages/HomePage.tsx	useFetch	GET /api/sessions?status=Scheduled&limit=4&sort=date&order=asc	routers/sessions.router.ts	aucun	sessions.controller.ts → getSessions

Categories

Page frontend	Fichier	Appel	Endpoint	Router	Middleware	Controller
CategoriesPage	pages/CategoriesPage.tsx	useFetch	GET /api/categories?limit=12&page=N	routers/categories.router.ts	aucun	categories.controller.ts → getCategories
CategoryDetailPage	pages/CategoryDetailPage.tsx	fetch natif	GET /api/categories/by-slug/:slug	routers/categories.router.ts	aucun	categories.controller.ts → getCategoryBySlug
CategoryDetailPage	pages/CategoryDetailPage.tsx	fetch natif	GET /api/activities?category_slug=X&limit=12	routers/activities.router.ts	aucun	activities.controller.ts → getActivities
HomePage	pages/HomePage.tsx	useFetch	GET /api/categories?limit=4	routers/categories.router.ts	aucun	categories.controller.ts → getCategories
DynamicDetailPage	pages/DynamicDetailPage.tsx	fetch natif	GET /api/categories/by-slug/:slug	routers/categories.router.ts	aucun	categories.controller.ts → getCategoryBySlug

Auth (public — non authentifié)

Page frontend	Fichier	Appel	Endpoint	Router	Middleware	Controller
LoginPage (formulaire login)	pages/LoginPage.tsx	store.login() → fetch	POST /api/auth/login	routers/auth.router.ts	aucun	auth.controller.ts → loginUser
LoginPage (formulaire login)	pages/LoginPage.tsx	store.login() → fetch	GET /api/auth/profile	routers/auth.router.ts	requireAuth	auth.controller.ts → getAuthenticatedUser
LoginPage (formulaire register)	pages/LoginPage.tsx	store.register() → fetch	POST /api/auth/register	routers/auth.router.ts	aucun	auth.controller.ts → registerUser
App.tsx (montage)	components/App.tsx	store.refreshToken() → fetch	POST /api/auth/refresh	routers/auth.router.ts	aucun (lit cookie httpOnly)	auth.controller.ts → refreshToken
App.tsx (montage, si refresh OK)	components/App.tsx	store.refreshToken() → fetch	GET /api/auth/profile	routers/auth.router.ts	requireAuth	auth.controller.ts → getAuthenticatedUser

Zone 2 — Espace client (/dashboard, /dashboard/commandes/:id)

Auth (authentifié)

Page frontend	Fichier	Appel	Endpoint	Router	Middleware	Controller
DashboardPage (bouton déconnexion)	pages/dashboard/DashboardPage.tsx	store.logout() → fetch	POST /api/auth/logout	routers/auth.router.ts	aucun	auth.controller.ts → logoutUser
DashboardPage (guard bfcache)	pages/dashboard/DashboardPage.tsx	useEffect + useNavigate	—	—	—	redirect /login si !user && !isHydrating

Page frontend	Fichier	Appel	Endpoint	Router	Middleware	Controller
OrderDetailPage (guard bfcache)	pages/dashboard/OrderDetailPage.tsx	useEffect → redirect si !user	—	—	—	—

Orders

Page frontend	Fichier	Appel	Endpoint	Router	Middleware	Controller
DashboardPage	pages/dashboard/DashboardPage.tsx	apiFetch	GET /api/orders/mine	routers/orders.router.ts	requireAuth + requireRole('member', 'admin')	orders.controller.ts → getMyOrders
OrderDetailPage	pages/dashboard/OrderDetailPage.tsx	apiFetch	GET /api/orders/:id	routers/orders.router.ts	requireAuth + requireRole('member', 'admin')	orders.controller.ts → getOrder
OrderDetailPage (payer)	pages/dashboard/OrderDetailPage.tsx	apiFetch	PUT /api/orders/:id { status: 'Confirmed' }	routers/orders.router.ts	requireAuth + requireRole('member', 'admin')	orders.controller.ts → updateOrder
OrderDetailPage (annuler)	pages/dashboard/OrderDetailPage.tsx	apiFetch	PUT /api/orders/:id { status: 'Cancelled' }	routers/orders.router.ts	requireAuth + requireRole('member', 'admin')	orders.controller.ts → updateOrder
BasketPage (commander)	pages/BasketPage.tsx	apiFetch	POST /api/orders	routers/orders.router.ts	requireAuth + requireRole('member', 'admin')	orders.controller.ts → createOrder

Zone 3 — Backoffice admin (/manage)

Page frontend	Fichier	Appel	Endpoint	Router	Middleware	Controller
ManageActivitiesPage	pages/manage/ManageActivitiesPage.tsx	apiFetch	GET /api/activities	activities.router.ts	requireAuth + requireRole('admin')	getActivities
ManageActivityFormPage	pages/manage/ManageActivityFormPage.tsx	apiFetch	POST /api/activities	activities.router.ts	requireAuth + requireRole('admin')	createActivity
ManageActivityFormPage	pages/manage/ManageActivityFormPage.tsx	apiFetch	PUT /api/activities/:id	activities.router.ts	requireAuth + requireRole('admin')	updateActivity
ManageActivitiesPage	pages/manage/ManageActivitiesPage.tsx	apiFetch	DELETE /api/activities/:id	activities.router.ts	requireAuth + requireRole('admin')	deleteActivity
ManageSessionsPage	pages/manage/ManageSessionsPage.tsx	apiFetch	GET /api/sessions	sessions.router.ts	requireAuth + requireRole('admin')	getSessions
ManageSessionFormPage	pages/manage/ManageSessionFormPage.tsx	apiFetch	POST /api/sessions	sessions.router.ts	requireAuth + requireRole('admin')	createSession
ManageSessionFormPage	pages/manage/ManageSessionFormPage.tsx	apiFetch	PUT /api/sessions/:id	sessions.router.ts	requireAuth + requireRole('admin')	updateSession

Page frontend	Fichier	Appel	Endpoint	Router	Middleware	Controller
ManageSessionPage	pages/manage/ManageSessionPage.tsx	apiFetch	DELETE /api/sessions/:id	sessions.router.ts	requireAuth + requireRole('admin')	deleteSession
ManageCategoriesPage	pages/manage/ManageCategoriesPage.tsx	apiFetch	GET /api/categories	categories.router.ts	requireAuth + requireRole('admin')	getCategories
ManageCategoryFormPage	pages/manage/ManageCategoryFormPage.tsx	apiFetch	POST /api/categories	categories.router.ts	requireAuth + requireRole('admin')	createCategory
ManageCategoryFormPage	pages/manage/ManageCategoryFormPage.tsx	apiFetch	PUT /api/categories/:id	categories.router.ts	requireAuth + requireRole('admin')	updateCategory
ManageCategoriesPage	pages/manage/ManageCategoriesPage.tsx	apiFetch	DELETE /api/categories/:id	categories.router.ts	requireAuth + requireRole('admin')	deleteCategory
ManageOrdersPage	pages/manage/ManageOrdersPage.tsx	apiFetch	GET /api/orders	orders.router.ts	requireAuth + requireRole('admin')	getOrders
ManageUsersPage	pages/manage/ManageUsersPage.tsx	apiFetch	GET /api/users	users.router.ts	requireAuth + requireRole('admin')	getUsers

Observations et anomalies

#	Fichier(s) concerné(s)	Type	Description
1	pages/ActivityDetailPage.tsx, pages/SessionDetailPage.tsx, pages/CategoryDetailPage.tsx, pages/DynamicDetailPage.tsx	Incohérence	Ces pages utilisent <code>fetch</code> natif sans <code>credentials: 'include'</code> . Endpoints ciblés publics — pas de bug actuellement. À remplacer par <code>apiFetch</code> si l'un d'eux devient protégé.
2	hooks/useFetch.ts	Incohérence	<code>useFetch</code> n'utilise pas <code>credentials: 'include'</code> et ne gère pas les 401. Réserve aux routes publiques uniquement.
3	store/basketStore.ts	~~Incomplet~~	~~Flux panier → création commande manquant.~~ ■ Résolu — <code>BasketPage</code> implémentée, <code>POST /api/orders</code> câblé.
4	pages/dashboard/DashboardPage.tsx	~~Sécurité~~	~~Guard <code>bfcache</code> absent sur <code>DashboardPage</code> .~~ ■ Résolu (<code>global-audit-and-fixes</code>) — <code>useEffect</code> + <code>useNavigate('/login', { replace: true })</code> ajouté.
5	store/authStore.ts + App.tsx	~~Sécurité~~	~~ <code>persist</code> Zustand stocke <code>user</code> (PII) dans <code>localStorage</code> .~~ ■ Résolu (<code>global-audit-and-fixes</code>) — <code>persist</code> supprimé, <code>localStorage.removeItem('zz-auth')</code> au montage de <code>App.tsx</code> .

#	Fichier(s) concerné(s)	Type	Description
6	<code>store/basketStore.ts</code>	Sécurité mineure	<code>basketStore</code> persiste tout le store en <code>localStorage</code> (pas de <code>partialize</code>). Données panier modifiables par XSS, mais validation backend protège (capacity + prix recalculé en BDD). Non bloquant.
7	<code>controllers/orders.controller.ts</code>	TODO	<code>updateOrder</code> (statut → <code>Confirmed</code>) simule un paiement sans Stripe. Hors MVP — documenté.
8	<code>pages/</code> (manage)	~~Manquant~~	~~Aucune page <code>/manage</code> implémentée frontend.~~ ■ Résolu (<code>admin-backoffice-dev</code>) — CRUD complet activités, sessions, catégories, orders, users.
9	<code>docker/nginx.conf</code>	■ Résolu (2026-06-08)	Headers de sécurité HTTP ajoutés : <code>X-Frame-Options: DENY</code> , <code>X-Content-Type-Options: nosniff</code> , <code>Strict-Transport-Security</code> , <code>Referrer-Policy</code> .
B1	<code>controllers/orders.controller.ts</code>	~~Sécurité~~	~~ <code>user_id</code> venait du body — n'importe quel membre pouvait créer une commande au nom d'autrui.~~ ■ Résolu (<code>global-audit-and-fixes</code>) — <code>user_id</code> retiré du schema Zod, remplacé par <code>req.user.id</code> .
B2	<code>controllers/users.controller.ts</code>	~~Incohérence~~	~~ <code>deleteUser</code> faisait un hard delete (<code>prisma.users.delete</code>).~~ ■ Résolu (<code>global-audit-and-fixes</code>) — soft delete { <code>deleted_at: new Date()</code> }.
B3	<code>controllers/users.controller.ts</code>	~~Robustesse~~	~~ <code>updateUser</code> sans validation Zod sur le body.~~ ■ Résolu (<code>global-audit-and-fixes</code>) — schema Zod ajouté, vérification existence rôle en BDD, <code>ZodError</code> géré.
B4	<code>controllers/auth.controller.ts</code>	~~Sécurité~~	~~ <code>persistRefreshToken</code> hashait le <code>tokenId</code> (UUID) plutôt que le JWT.~~ ■ Résolu (PR #40 <code>fix/refresh-token-hash-jwt</code>) — <code>hashPassword(jwt)</code> + <code>comparePassword(raw, stored.token_hash)</code> .

PARTIE C — Diagrammes ASCII par zone

Légende :

- → flux de données
- [MW] middleware

→ controllers/sessions.controller.ts → getSessions()

CATEGORIES

CategoriesPage.tsx

→ useFetch('/api/categories?limit=12&page=N')
→ GET /api/categories (public)
→ routers/categories.router.ts
→ controllers/categories.controller.ts → getCategories()
→ PostgreSQL : categories + activities_categories + activities

CategoryDetailPage.tsx

→ fetch('/api/categories/by-slug/:slug') [fetch natif]
→ GET /api/categories/by-slug/:slug (public)
→ controllers/categories.controller.ts → getCategoryBySlug()
→ PostgreSQL

CategoryDetailPage.tsx (activités de la catégorie)

→ fetch('/api/activities?category_slug=X&limit=12') [fetch natif]
→ GET /api/activities (public)
→ controllers/activities.controller.ts → getActivities()
→ PostgreSQL

HomePage.tsx

→ useFetch('/api/categories?limit=4')
→ GET /api/categories (public)
→ controllers/categories.controller.ts → getCategories()

AUTH – flux de démarrage (App.tsx)

App.tsx useEffect au montage

→ localStorage.removeItem('zz-auth') [nettoyage sessions persistées legacy]
→ store/authStore.ts → refreshToken()
→ [déduplication via refreshPromise – évite double appel React StrictMode]
→ fetch('/api/auth/refresh', { method: 'POST', credentials: 'include' })
→ POST /api/auth/refresh (lit cookie httpOnly refreshToken)
→ routers/auth.router.ts
→ controllers/auth.controller.ts → refreshAccessToken()
→ jwt.verify(raw) → extrait { userId, tokenId }
→ PostgreSQL : RefreshToken.findUnique({ token_id })
→ comparePassword(raw, stored.token_hash) [argon2 – vérifie le JWT contre le hash]
→ DELETE ancien token + INSERT nouveau token
→ Set-Cookie: accessToken (httpOnly, 15min) + refreshToken (httpOnly, 7j)
→ si OK : fetch('/api/auth/profile', { credentials: 'include' })
→ GET /api/auth/profile
→ [MW] requireAuth → lit cookie accessToken → req.user
→ controllers/auth.controller.ts → getAuthenticatedUser()
→ PostgreSQL : users (findUnique, omit password_hash)
→ store.set({ user, isHydrating: false })
→ si KO : store.set({ user: null, isHydrating: false })

AUTH – login

LoginPage.tsx formulaire login → handleLogin()

→ store/authStore.ts → login(email, password)
→ fetch('/api/auth/login', { method: 'POST', credentials: 'include', body: {email, password} })
→ POST /api/auth/login (public)
→ controllers/auth.controller.ts → loginUser()
→ lib/auth.ts → comparePassword() [argon2]
→ lib/tokens.ts → generateAccessToken() + generateRefreshToken()
→ persistRefreshToken(userId, tokenId, jwt)
→ PostgreSQL : RefreshToken.create { token_id, token_hash: hashPassword(jwt) }
[■ PR #40 – c'est le JWT qui est hashé, pas le tokenId]
→ Set-Cookie: accessToken + refreshToken
→ fetch('/api/auth/profile', { credentials: 'include' })
→ [MW] requireAuth

```
→ controllers/auth.controller.ts → getAuthenticatedUser()
→ store.set({ user })
```

AUTH – register

```

LoginPage.tsx formulaire register → handleRegister()
→ store/authStore.ts → register({ firstname, lastname, email, password, confirm })
→ fetch('/api/auth/register', { method: 'POST', credentials: 'include', body: {..., role_id: 1} })
→ POST /api/auth/register (public)
→ controllers/auth.controller.ts → registerUser()
→ lib/auth.ts → hashPassword() [argon2]
→ PostgreSQL : users.create
→ lib/tokens.ts → generateAccessToken() + generateRefreshToken()
→ persistRefreshToken(userId, tokenId, jwt) [hash argon2 du JWT]
→ Set-Cookie: accessToken + refreshToken
→ store.set({ user: data.data })
[note : register ne re-fetche pas /profile – user vient directement du body de réponse]

```

Zone 2 — Espace client (/dashboard)

AUTH – logout

```
DashboardPage.tsx bouton "Se déconnecter"
→ store/authStore.ts → logout()
→ fetch('/api/auth/logout', { method: 'POST', credentials: 'include' })
→ POST /api/auth/logout (public – pas de requireAuth)
→ controllers/auth.controller.ts → logoutUser()
→ verifyRefreshToken(raw) → userId
→ PostgreSQL : RefreshToken.deleteMany({ user_id })
→ finally : res.clearCookie('accessToken') + res.clearCookie('refreshToken')
→ store.set({ user: null })
```

GUARD bfcache – DashboardPage

```
DashboardPage.tsx useEffect au montage (+ sur restauration bfcache)
→ si isHydrating : attendre
→ si !user : navigate('/login', { replace: true })
[protection contre restauration bfcache après déconnexion]
```

ORDERS – liste

```
DashboardPage.tsx useEffect (déclenché quand user !== null)
→ store/authStore.ts → apiFetch('/api/orders/mine')
→ fetch('/api/orders/mine', { credentials: 'include' })
→ si 401 et !isRefreshing : apiFetch déclenche POST /api/auth/refresh → retry
[isRefreshing = flag module-level évite les boucles infinies sur 401 concurrent]
→ GET /api/orders/mine (auth)
→ routers/orders.router.ts
→ [MW] requireAuth → lit cookie accessToken → req.user.id
→ [MW] requireRole('member','admin') → PostgreSQL : roles.findUnique
→ controllers/orders.controller.ts → getMyOrders()
→ PostgreSQL : orders.findMany({ where: { user_id: req.user.id } })
include orders_lines + session + activity.title
→ { data: Order[] }
→ DashboardPage.tsx → setOrders(d.data)
```

ORDERS - détail

```
OrderDetailPage.tsx useEffect (déclenché au montage)
→ apiFetch('/api/orders/:id')
→ GET /api/orders/:id (auth)
→ [MW] requireAuth + requireRole('member','admin')
→ controllers/orders.controller.ts → getOrder()
→ PostgreSQL : orders.findUnique({ where: { id, user_id: req.user.id } })
[admin : sans filtre user id]
```

[illegible]

Zone 3 — Backoffice admin (/manage)

```
AUTH admin
```

```
ManageLoginPage.tsx formulaire login admin  
→ store/authStore.ts → login(email, password)  
→ POST /api/auth/login (public)  
→ Set-Cookie: accessToken + refreshToken  
→ si role !== 'admin' : logout() + erreur  
→ navigate('/manage')
```

```
AdminGuard (wrapper Route /manage)  
→ vérifie store.user?.role === 'admin'  
→ si non : redirect /manage/login
```

ACTIVITÉS – CRUD

```
ManageActivitiesPage.tsx  
→ apiFetch('/api/activities') GET (admin)  
→ apiFetch('/api/activities/:id', DELETE) (admin)  
→ [MW] requireAuth + requireRole('admin')  
→ controllers/activities.controller.ts → getActivities() / deleteActivity()  
→ deleteActivity() : refuse si orders liées ou sessions liées
```

```
ManageActivityFormPage.tsx (create / edit)
→ apiFetch('/api/activities', POST) ou apiFetch('/api/activities/:id', PUT)
→ [MW] requireAuth + requireRole('admin')
→ createActivity() : génère slug, crée activities_categories
→ updateActivity() : deleteMany pivot + recreate
```

SESSIONS — CRUD

ManageSessionsPage.tsx

```
→ apiFetch('/api/sessions') GET (admin)
→ apiFetch('/api/sessions/:id', DELETE) (admin)
→ deleteSession() : refuse si orders_lines liées
```

ManageSessionFormPage.tsx

```
→ apiFetch('/api/sessions', POST) ou apiFetch('/api/sessions/:id', PUT)
→ createSession() : valide date dans le futur
```

CATÉGORIES — CRUD

ManageCategoriesPage.tsx

```
→ apiFetch('/api/categories') GET (admin)
→ apiFetch('/api/categories/:id', DELETE) (admin)
→ deleteCategory() : refuse si activités orphelines (une seule catégorie)
```

ManageCategoryFormPage.tsx

```
→ apiFetch('/api/categories', POST) ou apiFetch('/api/categories/:id', PUT)
```

COMMANDES — liste admin

ManageOrdersPage.tsx

```
→ apiFetch('/api/orders') GET (admin uniquement — toutes commandes sans filtre user)
→ controllers/orders.controller.ts → getOrders()
```

UTILISATEURS — liste admin

ManageUsersPage.tsx

```
→ apiFetch('/api/users') GET (admin)
→ apiFetch('/api/users/:id', PUT) — updateUser() avec Zod + vérif rôle [■ corrigé #B3]
→ apiFetch('/api/users/:id', DELETE) — soft delete { deleted_at } [■ corrigé #B2]
```

PARTIE B — Flux narratif par type de données

1. Activities

Lecture publique (liste paginée)

ActivitiesPage et HomePage utilisent hooks/useFetch.ts qui émet un fetch natif sans credentials. L'URL /api/activities atterrit dans routers/activities.router.ts sans passer par aucun middleware. activities.controller.ts → getActivities() valide les query params via Zod (page, limit), puis interroge Prisma avec un include large : activities_categories → category, sessions → orders_lines → order → user. La réponse est reshapée pour exposer categories[] et sessions[] avec les users bookés.

Lecture publique (détail par slug)

ActivityDetailPage et DynamicDetailPage utilisent fetch natif direct (pas useFetch). getActivityBySlug() fait un findFirst({ where: { slug } }) avec le même include. En parallèle, ActivityDetailPage émet un second fetch vers /api/sessions?activity_slug=X&status=Scheduled pour lister les sessions disponibles de cette activité.

Écriture admin (create / update / delete)

ManageActivityFormPage et ManageActivitiesPage consomment ces endpoints via apiFetch. createActivity() génère un slug via utils/slugify.ts et crée les liaisons activities_categories en cascade. updateActivity() fait un deleteMany sur la table pivot puis recrée les liaisons. deleteActivity() refuse la

suppression si des `orders_lines` ou `sessions` sont liées.

2. Sessions

Lecture publique (liste avec filtres)

`SessionsPage` passe `status`, `sort`, `order` en query params. `getSessions()` valide via Zod un schéma strict (`SessionStatus` enum, `sort` enum `date|id`). Le calcul de `available_capacity` se fait en mémoire : `capacity - SUM(orders_lines.tickets_qty)` où les commandes `Cancelled` et `Refunded` sont exclues.

Lecture publique (détail)

`SessionDetailPage` appelle `GET /api/sessions/:id`. `getSession()` retourne le même calcul de capacité disponible, plus les données de l'activité associée (select minimal : `id`, `title`, `slug`, `image_filename`).

Écriture admin

`ManageSessionFormPage` crée et modifie. `createSession()` valide que la date est dans le futur. `deleteSession()` refuse si des `orders_lines` sont liées à la session.

3. Categories

Lecture publique

`CategoriesPage` → `getCategories()` : retourne `activities_categories` → `activity` (`id`, `title`, `slug` uniquement) + `activities_count`. `CategoryDetailPage` charge la catégorie par slug, puis en parallèle les activités via `/api/activities?category_slug=X`. `DynamicDetailPage` tente d'abord un fetch activité par slug, et si 404 tente un fetch catégorie par slug — route fourre-tout pour `/:slug`.

Écriture admin

`ManageCategoryFormPage` crée et modifie. `deleteCategory()` contient une protection spécifique : elle refuse si des activités liées n'ont qu'une seule catégorie (elles deviendraient orphelines).

4. Orders

Création (membre)

`BasketPage` appelle `POST /api/orders` via `apiFetch.createOrder()` ignore le `user_id` du body et utilise `req.user.id` (injecté par `requireAuth`). La transaction Prisma sérialisée crée la commande vide, vérifie la capacité atomiquement pour chaque ligne, calcule le total TTC, et retourne la commande complète. `BasketPage` redirige vers `OrderDetailPage` après succès.

Lecture (membre — mes commandes)

`DashboardPage` déclenche `apiFetch('/api/orders/mine')` uniquement quand `user !== null`. `apiFetch` gère le refresh automatique sur 401 via le flag module-level `isRefreshing` — évite les boucles infinies si plusieurs 401 arrivent simultanément. `getMyOrders()` filtre par `user_id: req.user.id`, retourne les lignes avec session + activity title.

Lecture (membre — détail)

`OrderDetailPage` appelle `apiFetch('/api/orders/:id')`. `getOrder()` filtre par `user_id: req.user.id` pour les membres (les admins voient tout).

Mutation de statut (membre)

`OrderDetailPage` émet `PUT /api/orders/:id` avec `{ status: 'Confirmed' }` (paiement simulé) ou `{ status: 'Cancelled' }`. `updateOrder()` applique une table de transitions autorisées (`VALID_TRANSITIONS`) : `Pending` → `Confirmed`, `Pending` → `Cancelled`, `Confirmed` → `Refunded`.

Lecture admin (liste complète)

`ManageOrdersPage` consomme `GET /api/orders` — réservé `admin`, retourne toutes les commandes sans filtre user.

5. Auth

Démarrage de l'application

`App.tsx` appelle `localStorage.removeItem('zz-auth')` pour nettoyer les sessions persistées legacy, puis `refreshToken()` dans un `useEffect` au montage. La déduplication via `refreshPromise` (variable module-level) évite le double appel en React StrictMode. `refreshAccessToken()` lit le cookie `httpOnly refreshToken`, extrait `tokenId` via `jwt.verify`, cherche en base par `token_id`, vérifie le JWT brut contre `token_hash` via `comparePassword(raw, stored.token_hash)` [argon2 — ■ PR #40 : c'est le JWT qui est hashé], puis fait un `deleteMany` atomique. Un nouveau pair `accessToken` / `refreshToken` est émis.

Login

`loginUser()` valide email + password via Zod, compare via argon2, génère access + refresh tokens, persiste le refresh en BDD via `persistRefreshToken(userId, tokenId, jwt)` — hash argon2 du JWT [■ PR #40], pose les deux cookies `httpOnly`. Le frontend enchaîne avec `GET /api/auth/profile`.

Register

Identique au login côté cookies. Le `user` est extrait du body de réponse sans re-fetch profile. Le `role_id: 1` est hardcodé côté frontend.

Logout

`logoutUser()` supprime tous les `RefreshToken` de l'utilisateur en BDD, puis `clearCookie` dans `finally`. Le frontend set `user: null`.

Guard bfcache (DashboardPage)

`DashboardPage` a un `useEffect` qui appelle `navigate('/login', { replace: true })` si `!user && !isHydrating`. Ce guard se re-déclenche sur restauration bfcache, contrairement au render-time `<Navigate>` qui lui est conservé pour le premier rendu synchrone.

Intercepteur 401 (apiFetch)

Sur 401, si `isRefreshing` est false : déclenche `POST /api/auth/refresh`, si OK retry la requête originale, si KO appelle `logout()` et redirige vers `/login`. Le flag `isRefreshing` est remis à false dans le bloc `finally`.

6. Panier (basketStore)

`basketStore` (Zustand + persist `localStorage`) gère les items localement : `addItem`, `removeItem`, `updateQuantity`, `clearBasket`, `totalItems()`, `totalPrice()`. `SessionDetailPage` appelle `addItem()` puis redirige vers `/panier`. `BasketPage` consomme le store et émet `POST /api/orders` via `apiFetch` au clic "Commander". Après succès : `clearBasket()` + redirect vers `OrderDetailPage`.

Note : `basketStore` persiste tout le store en `localStorage` (anomalie #6 ouverte — non bloquant, validation backend protège).

7. Infrastructure & sécurité

Headers de sécurité HTTP (■ ajouté 2026-06-08)

`docker/nginx.conf` — ajout de 4 headers sur toutes les réponses HTTPS :

- `X-Frame-Options: DENY` — empêche l'intégration dans une iframe (clickjacking)
- `X-Content-Type-Options: nosniff` — empêche le navigateur de deviner le type MIME
- `Strict-Transport-Security: max-age=31536000; includeSubDomains` — force HTTPS pour 1 an
- `Referrer-Policy: strict-origin-when-cross-origin` — limite les infos transmises dans le header Referer

Logs disponibles

- Logs Nginx : access log + error log (`/var/log/nginx/`) — automatiques
- Logs Docker : `docker compose logs backend` — capture stdout/stderr du container Node
- Logs applicatifs : `console.error` dans les blocs catch des controllers

Fin du document — mis à jour le 2026-06-08, branche `master`.